ContactUs.java

```
package Footer;

import java.time.Duration;

import org.openqa.selenium.By;
import org.openqa.selenium.JavascriptExecutor;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.Select;
import org.openqa.selenium.support.ui.WebDriverWait;

public class ContactUs {

   WebDriver driver;
   WebDriverWait wait;

   public ContactUs(WebDriver driver) {

      this.driver = driver;

      wait = new WebDriverWait(
            driver,
            Duration.ofSeconds(30)
      );
   }

   // Locators
   private By firstNameField = By.id("form-field-name");
   private By lastNameField = By.id("form-field-field_1c30d68");
   private By emailField = By.id("form-field-email");
   private By phoneField = By.id("form-field-phone");
   private By dropdownField = By.id("form-field-field_3d0acbe");
   private By messageField = By.id("form-field-message");
   private By submitButton = By.cssSelector("button.elementor-button[type='submit']");


   // Clear + fill helper
   private void typeInto(By locator, String value) {

      WebElement field = wait.until(
            ExpectedConditions.visibilityOfElementLocated(locator)
      );

      field.clear();

      if (value != null && !value.isEmpty()) {
         field.sendKeys(value);
      }
   }

   public void enterFirstName(String value) {
      typeInto(firstNameField, value);
   }
```

```
public void enterLastName(String value) {
   typeInto(lastNameField, value);
}

public void enterEmail(String value) {
   typeInto(emailField, value);
}

public void enterPhone(String value) {
   typeInto(phoneField, value);
}

public void enterMessage(String value) {
   typeInto(messageField, value);
}

// Select dropdown by visible text ("" for blank/default option)
public void selectQueryType(String visibleText) {

   WebElement dropdown = wait.until(
         ExpectedConditions.visibilityOfElementLocated(dropdownField)
   );

   Select select = new Select(dropdown);

   if (visibleText == null || visibleText.isEmpty()) {
      select.selectByIndex(0);
   } else {
      select.selectByVisibleText(visibleText);
   }
}

// Fill entire form in one call
public void fillForm(
      String firstName,
      String lastName,
      String email,
      String phone,
      String queryType,
      String message) {

   enterFirstName(firstName);
   enterLastName(lastName);
   enterEmail(email);
   enterPhone(phone);
   selectQueryType(queryType);
   enterMessage(message);
}

public void clickSubmit() {

   WebElement button = wait.until(
         ExpectedConditions.elementToBeClickable(submitButton)
   );

   ((JavascriptExecutor) driver).executeScript(
```

```
            "arguments[0].scrollIntoView({block:'center'});",
            button
        );

        button.click();
    }

    // Returns browser's native HTML5 validation message for a field
    // Empty string means the field is currently valid
    public String getValidationMessage(By fieldLocator) {

        WebElement field = driver.findElement(fieldLocator);

        return (String) ((JavascriptExecutor) driver)
                .executeScript("return arguments[0].validationMessage;", field);
    }

    public String getFirstNameValidationMessage() {
        return getValidationMessage(firstNameField);
    }

    public String getLastNameValidationMessage() {
        return getValidationMessage(lastNameField);
    }

    public String getEmailValidationMessage() {
        return getValidationMessage(emailField);
    }

    public String getPhoneValidationMessage() {
        return getValidationMessage(phoneField);
    }

    public String getMessageValidationMessage() {
        return getValidationMessage(messageField);
    }

    // Checks whether field currently satisfies HTML5 constraints
    public boolean isFieldValid(By fieldLocator) {

        WebElement field = driver.findElement(fieldLocator);

        return (boolean) ((JavascriptExecutor) driver)
                .executeScript("return arguments[0].checkValidity();", field);
    }

    public By getFirstNameLocator() { return firstNameField; }
    public By getLastNameLocator() { return lastNameField; }
    public By getEmailLocator() { return emailField; }
    public By getPhoneLocator() { return phoneField; }
    public By getMessageLocator() { return messageField; }
// Custom validation: name should contain only letters and spaces
    public boolean isNameFormatValid(String value) {
        return value != null && value.matches("^[A-Za-z ]+$");
    }

    // Custom validation: email format per common RFC-lite rules
```

```
  // (letters, numbers, . _ - +; no leading/trailing dot; no consecutive dots; max 64 chars local part)
  public boolean isEmailFormatValid(String email) {

    if (email == null || email.isEmpty()) {
      return false;
    }

    int atIndex = email.indexOf('@');
    if (atIndex <= 0) {
      return false;
    }

    String localPart = email.substring(0, atIndex);

    if (localPart.length() > 64) {
      return false;
    }

    if (localPart.startsWith(".") || localPart.endsWith(".")) {
      return false;
    }

    if (localPart.contains("..")) {
      return false;
    }

    return localPart.matches("^[A-Za-z0-9._+-]+$");
  }


}
```